# P R I S M

Technical Specification

---

ERD • System Architecture • API Design • User Flows

Backend: Python (FastAPI) • Frontend: Web (React/Next.js)

# 1. System Architecture

Prism follows a layered architecture with clear separation between the client tier, API layer, backend services, data stores, and external integrations.

## 1.1 Technology Stack

| Layer | Technology | Rationale |
|---|---|---|
| **API Framework** | FastAPI (Python 3.11+) | Async-first, auto-generated OpenAPI docs, Pydantic validation, fastest Python framework |
| **ORM** | SQLAlchemy 2.0 + Alembic | Mature, async support, excellent migration tooling |
| **Auth** | JWT (PyJWT) + OTP via SMS | Stateless tokens, OTP for phone-first Indian users |
| **Database** | PostgreSQL 16 | JSONB for flexible payloads, full-text search, battle-tested |
| **Cache** | Redis 7 | Session cache, streak tracking, rate limiting, pub/sub for real-time |
| **Task Queue** | Celery + Redis broker | Async jobs: card generation, PDF reports, notification dispatch |
| **AI Integration** | Anthropic Claude API (claude-sonnet-4-20250514) | Powers Ray mentor, mission scoring for open-ended challenges |
| **File Storage** | AWS S3 / MinIO | Prism Card images, PDF reports, career media |
| **Frontend** | Next.js 14 + Tailwind CSS + Framer Motion | SSR for SEO (landing page), PWA for app-like mobile experience |
| **Notifications** | WhatsApp Business API + Firebase Cloud Messaging | WhatsApp is primary channel for Indian teens and parents |
| **Hosting** | AWS India (ap-south-1) or GCP Mumbai | Low latency for Indian users, data residency compliance |
| **CI/CD** | GitHub Actions + Docker | Containerized deploys, automated testing |

## 1.2 Backend Service Modules

The FastAPI backend is organized as a modular monolith (not microservices for MVP). Each module is a separate router with its own models, schemas, and business logic. This allows splitting into microservices later without rewriting.

### `Module: auth/`

Handles OTP generation, verification, JWT issuance, and session management. Endpoints: POST /auth/otp/send, POST /auth/otp/verify, POST /auth/refresh, GET /auth/me. Redis stores OTP codes with 5-minute TTL. JWT tokens have 7-day expiry with refresh rotation.

### `Module: game/`

Manages worlds, missions, and attempts. Endpoints: GET /worlds, GET /worlds/{id}/missions, POST /missions/{id}/start, POST /missions/{id}/submit, GET /missions/{id}/result. Each mission submission triggers the identity engine recalculation via an internal event.

### `Module: identity/`

The trait scoring engine. Receives signals from mission attempts, This-or-That responses, and time-spent data. Recalculates the 5-dimension spectrum, updates archetype assignment, and records history. Endpoints: GET /spectrum/me, GET /spectrum/history, POST /spectrum/recalculate.

### `Module: ai/`

Powers the Ray AI mentor. Maintains conversation context per session. Injects the student's current spectrum and career bookmarks into the system prompt so Ray's responses are personalized. Endpoints: POST /ai/chat, GET /ai/conversations, GET /ai/conversations/{session_id}.

### `Module: careers/`

Career library CRUD and matching. Endpoints: GET /careers (filterable by stream, archetype, search), GET /careers/{id}, POST /careers/{id}/bookmark, GET /careers/recommended (based on current archetype). Admin endpoints for content management.

### `Module: cards/`

Prism Card generation. Takes current spectrum + archetype + badges + level and renders a card image (using Pillow or a headless browser rendering a React template). Endpoints: POST /cards/generate, GET /cards/{share_token} (public), GET /cards/me/history.

### `Module: school/`

School dashboard APIs. Endpoints: GET /school/dashboard (aggregate stats), GET /school/classes, GET /school/students, GET /school/students/{id}/profile, POST /school/reports/generate (triggers Celery job for PDF). Role-based access: only counsellor/admin roles.

### `Module: notifications/`

Dispatches daily quest reminders, streak alerts, and parent report notifications via WhatsApp Business API and web push. Uses Celery Beat for scheduled tasks (daily quest assignment at 7 AM IST, streak reminder at 6 PM IST).

# 2. Database Schema (ERD Detail)

The database uses PostgreSQL with 20 tables. All tables use UUID primary keys, timestamps, and soft-delete patterns where applicable. Below is the complete schema with column definitions, types, constraints, and indexes.

### Table: schools

School accounts and subscription info

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK, default uuid_generate_v4() |
| **name** | VARCHAR(255) | NOT NULL |
| **cbse_affiliation_no** | VARCHAR(50) | UNIQUE, indexed |
| **city** | VARCHAR(100) | NOT NULL |
| **state** | VARCHAR(100) | NOT NULL |
| **tier** | ENUM | 'starter','professional','premium' |
| **plan_type** | VARCHAR(50) | DEFAULT 'trial' |
| **student_limit** | INTEGER | DEFAULT 500 |
| **subscription_start** | DATE | |
| **subscription_end** | DATE | |
| **is_active** | BOOLEAN | DEFAULT true |
| **created_at** | TIMESTAMPTZ | DEFAULT now() |
| **updated_at** | TIMESTAMPTZ | DEFAULT now() |

### Table: users

All users: students, counsellors, admins, parents

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |
| **school_id** | UUID | FK → schools.id, nullable (anonymous quiz users) |
| **phone** | VARCHAR(15) | UNIQUE, indexed |
| **email** | VARCHAR(255) | nullable |
| **name** | VARCHAR(255) | NOT NULL |
| **role** | ENUM | 'student','counsellor','admin','parent' |

| | | |
|---|---|---|
| **avatar_url** | VARCHAR(500) | nullable |
| **xp_points** | INTEGER | DEFAULT 0 |
| **level** | INTEGER | DEFAULT 1 (1=Spark,2=Ray,3=Beam,4=Spectrum,5=Master) |
| **current_archetype_label** | VARCHAR(100) | nullable, denormalized for fast access |
| **onboarded** | BOOLEAN | DEFAULT false |
| **last_active_at** | TIMESTAMPTZ | nullable |
| **created_at** | TIMESTAMPTZ | DEFAULT now() |

### `Table: spectrums`

Core identity profile — one per student, continuously updated

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |
| **user_id** | UUID | FK → users.id, UNIQUE (one-to-one) |
| **analytical_creative** | FLOAT | Range -1.0 to 1.0 (-1 = analytical, +1 = creative) |
| **builder_explorer** | FLOAT | Range -1.0 to 1.0 |
| **leader_specialist** | FLOAT | Range -1.0 to 1.0 |
| **entrepreneur_steward** | FLOAT | Range -1.0 to 1.0 |
| **people_systems** | FLOAT | Range -1.0 to 1.0 |
| **primary_archetype_id** | UUID | FK → archetypes.id |
| **secondary_archetype_id** | UUID | FK → archetypes.id, nullable |
| **confidence_score** | FLOAT | 0.0–1.0, increases with more signals |
| **color_ratios** | JSONB | {"violet":0.3,"magenta":0.1,"cyan":0.25,"amber":0.2,"green":0.15} |
| **total_signals** | INTEGER | Count of all trait-signal events processed |
| **updated_at** | TIMESTAMPTZ | DEFAULT now() |

### `Table: spectrum_history`

Tracks how identity evolves over time

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |
| **spectrum_id** | UUID | FK → spectrums.id |
| **dimension_snapshot** | JSONB | Full copy of all 5 dimension scores at this point |
| **archetype_label** | VARCHAR(100) | Archetype at this point in time |

| | | |
|---|---|---|
| **confidence** | FLOAT | Confidence at this snapshot |
| **trigger_event** | VARCHAR(100) | 'mission_complete','tot_batch','daily_quest','manual_recalc' |
| **recorded_at** | TIMESTAMPTZ | DEFAULT now(), indexed |

## Table: archetypes

Archetype definitions (The Architect, The Storyteller, etc.)

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |
| **name** | VARCHAR(100) | UNIQUE, e.g. 'architect','storyteller','catalyst' |
| **label** | VARCHAR(100) | 'The Architect','The Storyteller', etc. |
| **description** | TEXT | 2–3 sentence personality description |
| **emoji_icon** | VARCHAR(10) | For UI display |
| **trait_ranges** | JSONB | Min/max ranges for each dimension that qualify for this archetype |
| **color_weights** | JSONB | Which spectrum colors dominate for this archetype |
| **is_active** | BOOLEAN | DEFAULT true |

## Table: worlds

The 3–5 game worlds

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |
| **name** | VARCHAR(100) | 'Space Lab','Creator Studio','Code Dungeon', etc. |
| **slug** | VARCHAR(50) | UNIQUE, URL-safe |
| **color_hex** | VARCHAR(7) | '#00ACC1' (Cyan for Space Lab), etc. |
| **description** | TEXT | World intro text shown to students |
| **icon_url** | VARCHAR(500) | World icon/illustration URL |
| **sort_order** | INTEGER | Display order |
| **is_active** | BOOLEAN | DEFAULT true |

## Table: missions

Challenges within worlds

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |

| | | |
|---|---|---|
| **world_id** | UUID | FK → worlds.id |
| **title** | VARCHAR(255) | e.g. 'Design a logo in 3 shapes' |
| **type** | ENUM | 'flash','scenario_sim','build_quest','ai_debate','this_or_that' |
| **difficulty** | ENUM | 'easy','medium','hard' |
| **duration_seconds** | INTEGER | Expected completion time |
| **content_payload** | JSONB | Mission-specific content (questions, prompts, assets) |
| **scoring_rubric** | JSONB | How to score responses (auto vs AI-graded) |
| **trait_weights** | JSONB | Which dimensions this mission affects and by how much |
| **is_active** | BOOLEAN | DEFAULT true |
| **sort_order** | INTEGER | Order within world |

## Table: mission_attempts

Every student attempt at a mission

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |
| **user_id** | UUID | FK → users.id, indexed |
| **mission_id** | UUID | FK → missions.id |
| **responses** | JSONB | Raw student responses/inputs |
| **scores** | JSONB | Detailed scoring breakdown |
| **creativity_score** | FLOAT | 0–100, AI-graded for creative missions |
| **speed_score** | FLOAT | 0–100, based on time vs expected |
| **accuracy_score** | FLOAT | 0–100, for objective missions |
| **xp_earned** | INTEGER | XP awarded for this attempt |
| **time_spent_sec** | INTEGER | Actual time spent |
| **status** | ENUM | 'in_progress','completed','abandoned' |
| **started_at** | TIMESTAMPTZ | |
| **completed_at** | TIMESTAMPTZ | nullable |

## Table: ai_conversations

Chat messages with Ray (AI mentor)

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |

| | | |
|---|---|---|
| **user_id** | UUID | FK → users.id, indexed |
| **session_id** | VARCHAR(100) | Groups messages into conversations, indexed |
| **role** | ENUM | 'user','assistant','system' |
| **content** | TEXT | Message content |
| **context_snapshot** | JSONB | Student's spectrum/archetype at time of message (for replay) |
| **tokens_used** | INTEGER | Token count for cost tracking |
| **created_at** | TIMESTAMPTZ | DEFAULT now() |

### Table: careers

Career library — 300+ careers with Indian context

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |
| **title** | VARCHAR(255) | 'Software Engineer','Chartered Accountant', etc. |
| **slug** | VARCHAR(100) | UNIQUE, URL-safe |
| **category** | VARCHAR(100) | 'Technology','Business','Healthcare', etc. |
| **stream_fit** | ENUM | 'science','commerce','humanities','any' |
| **description** | TEXT | 2–3 paragraph overview |
| **salary_range** | JSONB | {"entry":"4-8 LPA","mid":"12-25 LPA","senior":"30-60 LPA"} |
| **entry_paths** | JSONB | Array of educational pathways |
| **required_exams** | JSONB | JEE, NEET, CAT, CLAT, etc. |
| **college_options** | JSONB | Top colleges for this career |
| **day_in_life** | JSONB | Hour-by-hour description |
| **archetype_fit_ids** | UUID[] | Array of archetype IDs that best fit |
| **is_active** | BOOLEAN | DEFAULT true |

### Table: prism_cards

Generated shareable identity cards

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |
| **user_id** | UUID | FK → users.id, indexed |
| **card_version** | VARCHAR(20) | 'v1','v2' — tracks design iterations |
| **card_data** | JSONB | Snapshot of all data used to render the card |

| | | |
|---|---|---|
| **image_url** | VARCHAR(500) | S3 URL of rendered card image |
| **share_token** | VARCHAR(32) | UNIQUE, URL-safe token for public sharing |
| **is_public** | BOOLEAN | DEFAULT true |
| **generated_at** | TIMESTAMPTZ | DEFAULT now() |

### Table: otp_tokens

OTP codes for phone authentication

| Column | Type | Constraints / Notes |
|---|---|---|
| **id** | UUID | PK |
| **phone** | VARCHAR(15) | indexed |
| **otp_code** | VARCHAR(6) | 6-digit code |
| **attempts** | INTEGER | DEFAULT 0, max 3 |
| **is_used** | BOOLEAN | DEFAULT false |
| **expires_at** | TIMESTAMPTZ | now() + 5 minutes |
| **created_at** | TIMESTAMPTZ | DEFAULT now() |

### Additional tables (standard structure)

classes: school classes (grade, section, stream, academic_year). class_students: junction table linking users to classes. tot_questions: This-or-That question bank with trait weights per option. tot_responses: student responses with chosen option and response time. badges: badge definitions with unlock criteria as JSONB. user_badges: junction table with earned_at timestamp. career_bookmarks: junction between users and careers with optional notes. streaks: current and longest streak per user with last activity date. daily_quests: assigned mission per user per day with completion status. parent_reports: generated PDF reports per student with report data as JSONB.

## 2.1 Key Database Indexes

Beyond primary keys and foreign keys, these indexes are critical for performance:

```
CREATE INDEX idx_users_school_role ON users(school_id, role);
CREATE INDEX idx_users_phone ON users(phone);
CREATE INDEX idx_mission_attempts_user ON mission_attempts(user_id, completed_at DESC);
CREATE INDEX idx_spectrum_history_spectrum ON spectrum_history(spectrum_id, recorded_at DESC);
CREATE INDEX idx_ai_conversations_session ON ai_conversations(session_id, created_at);
CREATE INDEX idx_prism_cards_share ON prism_cards(share_token) WHERE is_public = true;
CREATE INDEX idx_daily_quests_user_date ON daily_quests(user_id, quest_date);
CREATE INDEX idx_careers_stream ON careers(stream_fit) WHERE is_active = true;
```

# 3. API Design

All APIs follow RESTful conventions with JSON request/response bodies. Authentication is via Bearer JWT token in the Authorization header. All timestamps are UTC in ISO 8601 format.

## 3.1 Authentication Endpoints

| Method | Endpoint | Payload | Notes |
|---|---|---|---|
| **POST** | /api/v1/auth/otp/send | {phone} | Sends 6-digit OTP via SMS/WhatsApp. Rate limited: 3/min per phone. |
| **POST** | /api/v1/auth/otp/verify | {phone, otp, name?} | Verifies OTP, creates user if new, returns {access_token, refresh_token, user}. |
| **POST** | /api/v1/auth/refresh | {refresh_token} | Returns new access_token. Refresh tokens are single-use. |
| **GET** | /api/v1/auth/me | Bearer token | Returns current user profile with spectrum summary. |

## 3.2 Game Endpoints

| Method | Endpoint | Payload | Notes |
|---|---|---|---|
| **GET** | /api/v1/worlds | — | Returns all active worlds with mission counts and user progress. |
| **GET** | /api/v1/worlds/{id}/missions | ?type=flash | Returns missions for a world, filterable by type. |
| **POST** | /api/v1/missions/{id}/start | — | Creates a mission_attempt record, returns mission content_payload. |
| **POST** | /api/v1/missions/{id}/submit | {responses, time_spent} | Scores attempt, awards XP, triggers identity engine recalc. |
| **POST** | /api/v1/tot/submit-batch | {responses: [{qid, choice}]} | Submits This-or-That responses in bulk (onboarding quiz). |

## 3.3 Identity & AI Endpoints

| Method | Endpoint | Payload | Notes |
|---|---|---|---|
| **GET** | /api/v1/spectrum/me | — | Returns full spectrum: dimensions, archetypes, confidence, color ratios. |

| | | | |
|---|---|---|---|
| **GET** | /api/v1/spectrum/history | ?limit=20 | Returns spectrum evolution timeline. |
| **POST** | /api/v1/ai/chat | {message, session_id?} | Send message to Ray. Returns AI response. Creates session if new. |
| **GET** | /api/v1/ai/conversations | ?limit=10 | Lists conversation sessions with preview. |
| **GET** | /api/v1/careers | ?stream=science&q=data | Search/filter career library. |
| **GET** | /api/v1/careers/recommended | — | Returns careers matching current archetype. |
| **POST** | /api/v1/careers/{id}/bookmark | {notes?} | Bookmark a career. |

## 3.4 Card & Sharing Endpoints

| Method | Endpoint | Payload | Notes |
|---|---|---|---|
| **POST** | /api/v1/cards/generate | — | Generates a new Prism Card image from current spectrum. Returns {image_url, share_token}. |
| **GET** | /api/v1/cards/me | — | Returns all cards generated by user, newest first. |
| **GET** | /api/v1/cards/{share_token} | — | PUBLIC (no auth). Returns card data + image for sharing page. |
| **GET** | /api/v1/cards/{share_token}/og | — | Returns OpenGraph metadata for rich link previews on WhatsApp/Instagram. |

## 3.5 School Dashboard Endpoints

| Method | Endpoint | Payload | Notes |
|---|---|---|---|
| **GET** | /api/v1/school/dashboard | — | Aggregate stats: total students, avg engagement, archetype distribution, stream readiness. |
| **GET** | /api/v1/school/classes | — | List classes with student counts and engagement metrics. |
| **GET** | /api/v1/school/students | ?class_id=&sort= | Paginated student list with spectrum summaries. |
| **GET** | /api/v1/school/students/{id} | — | Full student profile: spectrum, history, attempts, badges. |
| **POST** | /api/v1/school/reports/generate | {student_id?, class_id?} | Triggers async parent report PDF generation via Celery. |

# 4. Identity Engine Algorithm

## 4.1 Signal Processing

Every interaction generates trait signals. A signal is a weighted adjustment to one or more of the 5 dimensions. The identity engine processes signals in near-real-time (triggered after each mission submission or This-or-That batch).

### Signal sources and weights

This-or-That responses: direct, binary signal. Each question has predefined trait_weights_a and trait_weights_b (e.g., choosing option A in a question might add +0.15 to analytical_creative toward the creative end). Weight: HIGH (these are the most intentional self-reported signals).

Mission performance: indirect signal. The scoring rubric maps scores to trait adjustments. A high creativity_score on a Creator Studio mission adds to the creative dimension. A high speed_score on a Code Dungeon puzzle adds to the analytical dimension. Weight: MEDIUM (behavioral, less noisy than self-report).

Time-spent patterns: implicit signal. If a student spends significantly more time browsing careers in the business category, this nudges the entrepreneur dimension. Weight: LOW (noisy but useful at scale).

AI chat signals: Ray can detect preferences from conversation patterns. If a student repeatedly asks about design careers, this is a soft signal. Weight: LOW.

## 4.2 Dimension Update Formula

For each dimension d (on a -1.0 to +1.0 scale):

```
new_value = old_value + (signal_strength * learning_rate * decay_factor)
```

Where: signal_strength is the raw trait weight from the signal source (-1.0 to +1.0). learning_rate starts at 0.3 for new users (fast initial formation) and decays to 0.05 for established profiles (stability). decay_factor = 1 / log2(total_signals + 2), ensuring diminishing returns.

The value is clamped to [-1.0, +1.0] after update. After each update, the confidence_score is recalculated as min(1.0, total_signals / 50) — meaning full confidence requires ~50 signals (roughly 10 missions + 1 This-or-That batch).

## 4.3 Archetype Assignment

After each dimension update, the engine checks which archetype best fits the current spectrum. Each archetype has a trait_ranges definition in JSONB that specifies the ideal ranges for each dimension. The engine calculates a match score for each archetype using cosine similarity between the student's dimension vector and the archetype's ideal center point.

The primary archetype is the highest match score. The secondary archetype is the second-highest, but only assigned if its match score is above 0.6 (otherwise null). An archetype change only triggers if the new match score exceeds the current archetype's match score by at least 0.1 (hysteresis to prevent flip-flopping).

## 4.4 Color Ratio Calculation

The color_ratios field maps directly to the 5 game worlds. Each dimension contributes to its corresponding color: analytical_creative maps to Violet (analytical) and Magenta (creative). builder_explorer maps to Amber (builder) and Cyan (explorer). The people_systems dimension maps to Green. The ratios always sum to 1.0 and are used to render the spectrum bar on the Prism Card.

## 5. Prism Card Generation Pipeline

The Prism Card is the most important output of the entire platform. The generation pipeline must produce a visually stunning, shareable image in under 3 seconds.

### Step 1: Data Assembly

Collect: current spectrum (5 dimensions + color ratios), primary and secondary archetype (name, label, description), top 3 badges (icon + name), current level (1–5), XP points, total missions completed, and the student's name.

### Step 2: Template Rendering

A Next.js page at /card/[share_token] renders a React component with the card design. This page is server-side rendered with dynamic OG meta tags for rich link previews. The card component uses Tailwind CSS with the Prism color palette and is designed at 1080×1920 pixels (Instagram Story ratio).

### Step 3: Image Capture

For shareable images, a Celery worker uses Playwright (headless Chromium) to screenshot the rendered card page at 2x resolution. The image is uploaded to S3 and the URL is stored in the prism_cards table. Alternative for lower infra: use Pillow with a Python template to generate the card image directly without a browser.

### Step 4: Sharing

The public URL /card/[share_token] serves the card page with OG tags: og:image (the generated image), og:title ("I'm The Architect — what's your Prism?"), og:description (archetype description), and a CTA linking to the landing page. WhatsApp and Instagram will render a rich preview card when this URL is shared.

# 6. Project Structure

```
prism/
├── backend/
│   ├── app/
│   │   ├── main.py                # FastAPI app, middleware, CORS
│   │   ├── config.py              # Settings (env vars, DB URL, etc.)
│   │   ├── database.py            # SQLAlchemy engine + session
│   │   ├── dependencies.py        # get_current_user, get_db, etc.
│   │   ├── models/                # SQLAlchemy ORM models
│   │   │   ├── user.py
│   │   │   ├── school.py
│   │   │   ├── spectrum.py
│   │   │   ├── game.py
│   │   │   ├── career.py
│   │   │   └── card.py
│   │   ├── schemas/               # Pydantic request/response schemas
│   │   ├── routers/               # API route handlers
│   │   │   ├── auth.py
│   │   │   ├── game.py
│   │   │   ├── identity.py
│   │   │   ├── ai.py
│   │   │   ├── careers.py
│   │   │   ├── cards.py
│   │   │   └── school.py
│   │   ├── services/              # Business logic
│   │   │   ├── identity_engine.py  # Trait scoring algorithm
│   │   │   ├── ai_mentor.py        # Claude API integration
│   │   │   ├── card_generator.py   # Prism Card rendering
│   │   │   ├── otp_service.py      # SMS/WhatsApp OTP
│   │   │   └── notification.py     # Push + WhatsApp dispatch
│   │   └── tasks/                 # Celery async tasks
│   │       ├── card_tasks.py
│   │       ├── report_tasks.py
│   │       └── notification_tasks.py
│   ├── alembic/                   # Database migrations
│   ├── tests/
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/
```

```
│   ├── src/
│   │   ├── app/                    # Next.js App Router pages
│   │   │   ├── page.tsx            # Landing page (viral quiz)
│   │   │   ├── worlds/             # World map + missions
│   │   │   ├── card/[token]/       # Public shareable card
│   │   │   ├── dashboard/          # School admin dashboard
│   │   │   └── chat/               # Ray AI mentor interface
│   │   ├── components/             # Reusable React components
│   │   ├── hooks/                  # Custom hooks (useSpectrum, useAuth)
│   │   ├── lib/                    # API client, utils
│   │   └── styles/                 # Tailwind config + globals
│   └── Dockerfile
├── docker-compose.yml              # Full stack: api + db + redis + worker
└── seed/                           # Seed data scripts
    ├── archetypes.json
    ├── worlds.json
    ├── missions.json
    ├── tot_questions.json
    └── careers.json
```